

Relatório do WebAssembly

Perguntas:

1. Ao analisar o arquivo .wat gerado para a função 'soma', descreva o fluxo das instruções em comparação com o assembly MIPS ('add' / 'addu'). Como o WASM manipula as variáveis locais sem usar registradores como \$t0 ou \$t1?

Resposta:

O WASM, diferente do Assembly, trabalha com vetor de variáveis locais e máquina de pilha, esta é utilizada para realizar operações aritméticas, enquanto o vetor armazena os valores das variáveis baseadas nas operações da pilha.

Dessarte, as variáveis "a" e "b" funcionam como variáveis de parâmetro no array, de índices 0 e 1; enquanto são declaradas outras 6 variáveis locais para armazenar os valores temporariamente, compondo o array: [0, 1, 2, 3, 4, 5, 6, 7] (Análogo a \$t0 - \$t7).

A pilha inicia com o ponteiro de pilha, e então é realizado operações de subtração com ele, com o objetivo futuro de realizar leituras e armazenar seus valores na memória RAM. Assim, efetuando uma série de operações get e set para empilhar e desempilhar com retorno de/para valores/índices do array.

Além disso, são realizadas operações load/store com o offset fornecido (Com base em \$stack - 16) para armazenar e escrever na memória os resultados das leituras.

Ao final, realiza-se a soma de a + b na pilha (Desempilhando ambos e empilhando novamente a soma) e então retornando o topo da pilha, que é a + b, como retorno para a função.

2. No MIPS, um overflow na instrução 'add' gera uma exceção em hardware, enquanto 'addu' ignora silenciosamente. Como a sua função 'soma_segura' simula o comportamento da arquitetura? Foi mais custoso tratar via software (WASM/C)?

A soma do Webassembly funciona estritamente por addu, assim, não sinalizando overflow e descartando excesso de bits com wrap-around.

A função, assim como em C, verifica os bits de sinal para verificar se houve overflow com blocos br_if. Assim, sinalizando nível alto ao ponteiro de overflow, conforme expressão de Karnaugh obtida para essa condição.

Com certeza foi mais custoso implementar por software, pois, como se pôde analisar, a função Webassembly alocou 50 variáveis locais (Pouco mais que 8 vezes da sua versão insegura), além de implementar instruções branch e aumentar o número de instruções para aproximadamente 200.

3. Explique os valores do contraexemplo gerado pelo Map2Check. Como a representação em complemento de dois explica a falha identificada pela ferramenta?